

Parallelizing Conway’s Game of Life

Comparing Distributed-Memory and GPU Parallelism with MPI and CUDA

Krisnajt Rajeshkhanna

October – November 2024

Abstract

Conway’s Game of Life is an iterative cellular automaton in which each cell can be updated using only the state of its local neighborhood, making the workload well suited for studying parallel-computing techniques. This work investigates two approaches to parallelizing the simulation: distributed-memory execution across CPU processes using the Message Passing Interface (MPI), and massively parallel execution on a GPU using CUDA.

The MPI implementation partitions the simulation grid using one-dimensional row-wise domain decomposition and exchanges boundary rows between neighboring processes using non-blocking communication. The CUDA implementation maps cell updates onto a two-dimensional GPU thread hierarchy using custom kernels, padded grid representations, and two-dimensional grid-stride execution.

Performance experiments examine how MPI process count and CUDA thread-block configuration affect execution time on a 512×512 grid. MPI runtime decreases from approximately 0.0655 seconds at four processes to a minimum of 0.0090 seconds at 64 processes before runtime increases sharply at 128 processes as communication overhead grows relative to local computation. CUDA runtime decreases from 0.0357 seconds with 1×1 thread blocks to 0.00451 seconds with 8×8 blocks before larger configurations again reduce performance. Together, the experiments demonstrate that increasing parallelism does not guarantee monotonic performance improvements: distributed execution eventually enters a communication-limited scaling regime, while GPU execution is sensitive to launch configuration and hardware resource utilization.

1. Introduction

Conway’s Game of Life is a two-dimensional cellular automaton in which every cell is either alive or dead. At each generation, the next state of a cell is determined from the states of its eight neighboring cells. Since every update depends only on values from the previous generation, cells can be evaluated largely independently, exposing substantial data parallelism.

This structure makes Game of Life a useful workload for studying two different parallel execution models:

- **Distributed-memory parallelism**, where the global grid is partitioned across independent CPU processes and neighboring partitions communicate explicitly.
- **GPU parallelism**, where thousands of lightweight CUDA threads execute cell updates concurrently while sharing device memory.

Although both approaches increase computational parallelism, they introduce different bottlenecks. MPI execution must balance useful local computation against communication between processes. CUDA execution must instead balance thread-level parallelism against launch overhead, occupancy, and finite GPU resources.

This study therefore focuses on the following question:

How do distributed-memory and GPU parallelization strategies scale when applied to the same iterative grid computation, and what ultimately limits performance in each execution model?

Rather than treating the MPI and CUDA implementations as independent exercises, we analyze them as two approaches to exploiting the parallel structure of the same computational problem.

2. Computational Model

Let the Game of Life board at generation t be represented by

$$G^{(t)} \in \{0, 1\}^{H \times W},$$

where a value of 1 represents a live cell and 0 represents a dead cell.

For each cell (i, j) , define the number of live neighbors as

$$N_{i,j}^{(t)} = \sum_{a=-1}^1 \sum_{b=-1}^1 G_{i+a,j+b}^{(t)} - G_{i,j}^{(t)}.$$

The next state is determined by

$$G_{i,j}^{(t+1)} = \begin{cases} 1, & G_{i,j}^{(t)} = 1 \text{ and } N_{i,j}^{(t)} \in \{2, 3\}, \\ 1, & G_{i,j}^{(t)} = 0 \text{ and } N_{i,j}^{(t)} = 3, \\ 0, & \text{otherwise.} \end{cases}$$

Because every cell in generation $t + 1$ depends only on generation t , all cell updates within a generation can be performed concurrently as long as each worker has access to the required neighboring values.

This dependency structure motivates both the MPI and CUDA implementations.

3. MPI Implementation

3.1 Domain Decomposition

The MPI implementation distributes the simulation using a one-dimensional, row-wise decomposition. The root process initially reads the coordinates of live cells and constructs the global grid. Rows are then divided approximately evenly among the available MPI processes.

Each process is responsible for updating a contiguous subset of the board.

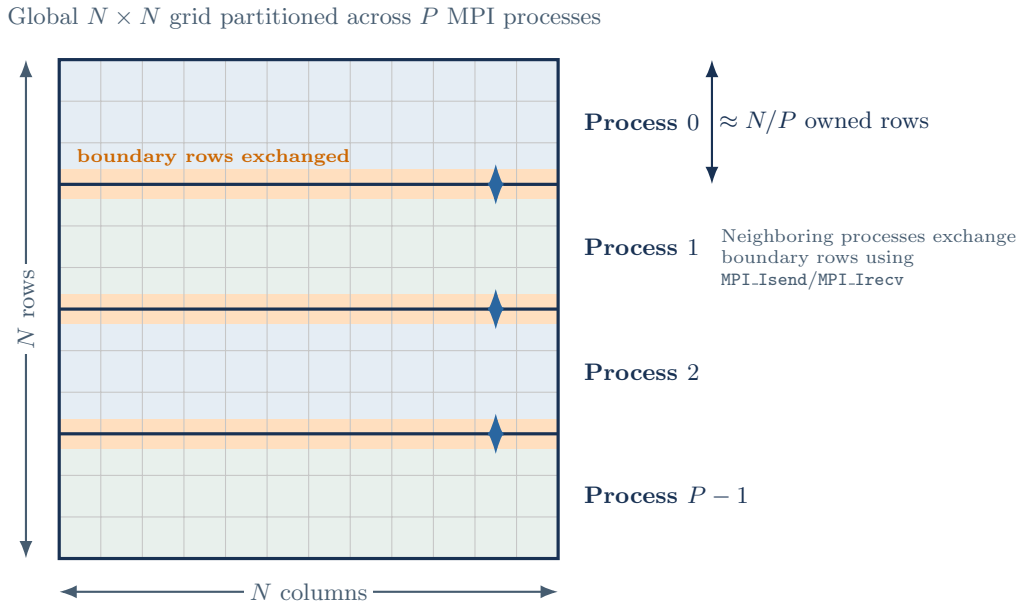


Figure 1: One-dimensional row-wise decomposition of an $N \times N$ Game of Life grid across P MPI processes. Each process owns approximately N/P rows; boundary rows are exchanged with adjacent processes and stored as local ghost data for neighbor-dependent updates. Non-blocking halo exchange allows communication to overlap with computation.

The decomposition reduces communication requirements because a process does not need the entire global grid. To update its local cells, it primarily needs its own rows together with boundary rows from adjacent partitions.

3.2 Non-Blocking Boundary Exchange

At each generation, neighboring processes exchange their boundary rows. Communication is implemented using the non-blocking MPI primitives

`MPI_Isend` and `MPI_Irecv`.

Unlike blocking communication, these operations allow transfers to begin without forcing the process to remain idle until they complete. A process can initiate boundary exchange and continue computing cells whose dependencies are already locally available.

Conceptually, each generation follows:

1. initiate asynchronous sends and receives for neighboring rows;
2. compute cells that do not depend on remote boundary values;
3. complete outstanding communication;
4. compute cells adjacent to partition boundaries;
5. advance to the next generation.

This approach attempts to hide part of the communication latency behind useful computation.

4. CUDA Implementation

4.1 GPU Representation

The CUDA implementation stores the two-dimensional board as a contiguous one-dimensional array in device memory. The host initializes the board and transfers the complete grid to the GPU using `cudaMemcpy` before iterative computation begins.

Keeping the simulation state on-device minimizes repeated CPU–GPU data movement during the computational portion of the simulation.

4.2 Kernel Organization

The implementation uses two CUDA kernels:

- `padded_matrix_copy`, which copies the current grid into a padded representation surrounded by dead cells;
- `compute_on_gpu`, which applies the Game of Life transition rules and computes the next generation.

Padding simplifies boundary handling by ensuring that cells along the original board edges can use the same neighborhood-access pattern as interior cells.

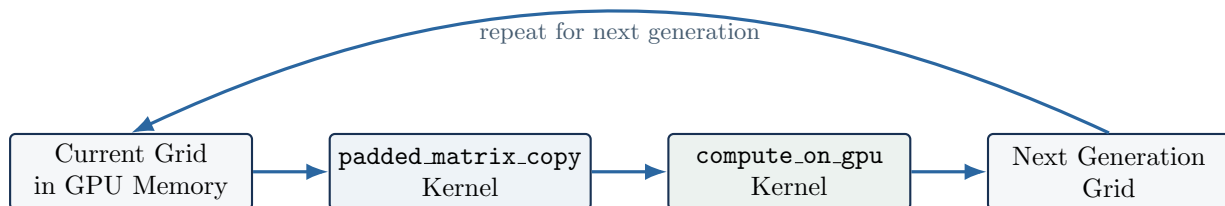


Figure 2: CUDA execution pipeline. A padded grid simplifies boundary handling before parallel cell updates compute the next generation.

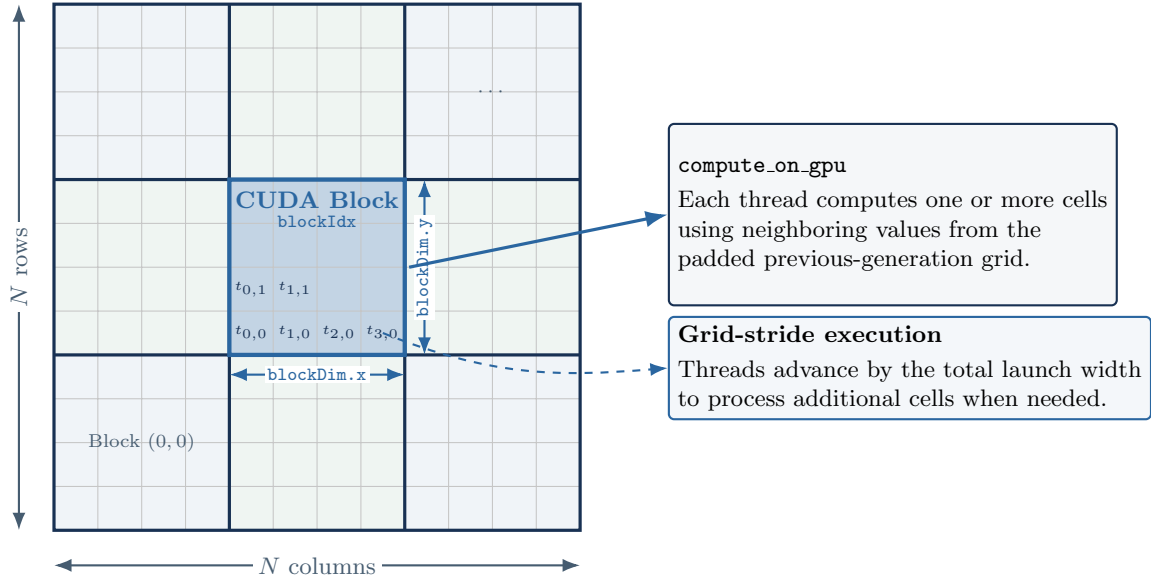
$N \times N$ Game of Life grid mapped onto a CUDA launch grid

Figure 3: Mapping of an $N \times N$ Game of Life board onto a two-dimensional CUDA execution grid. Each block launches a two-dimensional set of threads whose initial coordinates map to board cells; two-dimensional grid-stride loops allow threads to process additional cells when the board exceeds the launch dimensions.

4.3 Two-Dimensional Grid-Stride Execution

The CUDA kernels use a two-dimensional launch configuration in which both the thread blocks and the overall CUDA grid span the two dimensions of the Game of Life board. Each thread first computes its global coordinates as

$$i = \text{blockIdx.x} \times \text{blockDim.x} + \text{threadIdx.x},$$

$$j = \text{blockIdx.y} \times \text{blockDim.y} + \text{threadIdx.y}.$$

The total number of threads launched along each dimension is

$$S_x = \text{blockDim.x} \times \text{gridDim.x},$$

$$S_y = \text{blockDim.y} \times \text{gridDim.y}.$$

Each thread then traverses the board using a two-dimensional grid-stride loop:

$$i \leftarrow i + S_x, \quad j \leftarrow j + S_y.$$

In the implementation, these strides are realized through nested loops, allowing a thread to process multiple cells when the board dimensions exceed the number of simultaneously launched threads.

This mapping makes the kernel independent of a fixed board size while preserving the spatial correspondence between the two-dimensional simulation grid and the CUDA execution hierarchy.

5. Experimental Methodology

Both implementations were evaluated on a 512×512 simulation grid. However, the original experiments used different numbers of generations and therefore should be interpreted as independent scaling studies rather than a direct comparison of absolute MPI and CUDA runtime.

Table 1: Experimental configurations used in the MPI and CUDA studies.

Property	MPI Study	CUDA Study
Grid size	512×512	512×512
Generations	500	100
Variable	MPI process count	CUDA block dimension $n \times n$
Configurations	4, 8, 16, 32, 64, 128	$n = 1, 2, 4, 8, 16, 32$
Primary concern	Communication / decomposition	Thread scheduling / occupancy

The experiments are therefore used to answer two separate scaling questions:

1. How does increasing the number of MPI processes affect distributed simulation runtime?
2. How does CUDA thread-block dimension affect GPU execution time?

6. Results

6.1 MPI Scaling

Figure 4 shows the minimum, average, and maximum MPI runtime for process counts ranging from 4 to 128.

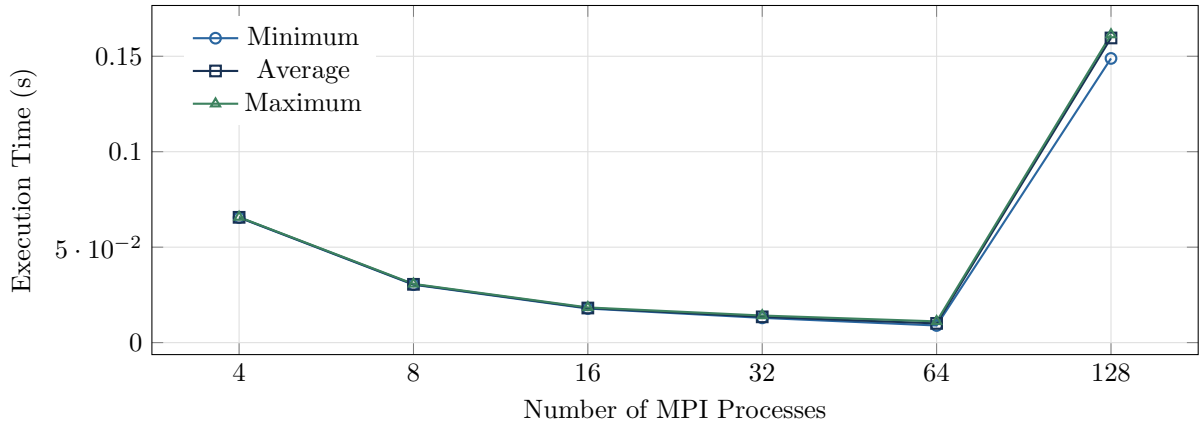


Figure 4: MPI execution time for 500 generations on a 512×512 grid. Performance improves through 64 processes before degrading sharply at 128 processes as communication overhead grows relative to local computation.

The minimum runtime decreases from

$$0.065490 \text{ s at 4 processes}$$

to

$$0.008970 \text{ s at 64 processes.}$$

Relative to the four-process configuration, this corresponds to

$$\frac{0.065490}{0.008970} \approx 7.30 \times$$

lower execution time.

The trend is not monotonic beyond this point. At 128 processes, minimum runtime increases sharply to 0.148856 seconds.

At high process counts, each process owns fewer rows and consequently performs less useful computation between communication events. Boundary exchanges, synchronization, and process-management overhead therefore become increasingly large relative to local computation.

The experiment demonstrates a characteristic strong-scaling limit: partitioning the workload further eventually produces insufficient computation per process to amortize distributed-memory communication.

6.2 CUDA Thread-Block Tuning

Figure 5 shows CUDA execution time as the square thread-block dimension n varies, where each block contains $n \times n$ threads.

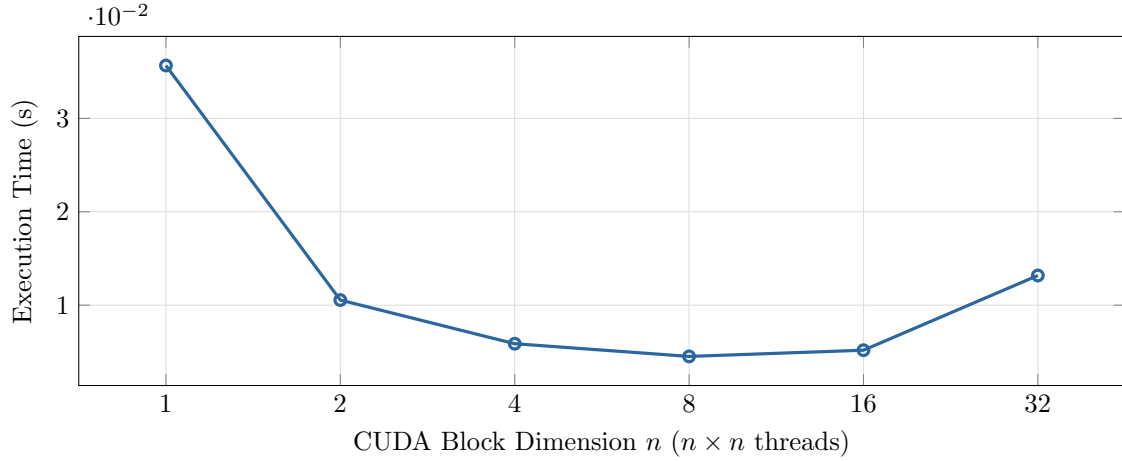


Figure 5: CUDA execution time for 100 generations on a 512×512 grid as the square thread-block dimension n is varied.

Execution time falls from

0.0356602 s with 1×1 thread blocks

to a minimum of

0.00451258 s with 8×8 thread blocks.

This represents a configuration-to-configuration reduction of

$$\frac{0.0356602}{0.00451258} \approx 7.90\times.$$

Increasing the block dimension beyond 8×8 does not continue to improve performance. Runtime increases to 0.0051767 seconds with 16×16 blocks and then to 0.0131774 seconds with 32×32 blocks.

The results indicate that increasing thread-level parallelism initially improves GPU utilization, but sufficiently large thread blocks reduce scheduling flexibility and may constrain occupancy depending on register usage and hardware limits. The optimal configuration in this experiment therefore lies between underutilization at small blocks and reduced execution efficiency at larger ones.

7. Comparative Analysis

Although MPI and CUDA expose parallelism through very different programming models, the experiments reveal a common performance principle:

Increasing available parallelism does not necessarily increase useful parallel performance.

Table 2 summarizes the dominant considerations observed in each implementation.

Table 2: Comparison of the two parallelization strategies.

Property	MPI	CUDA
Parallel unit	Independent CPU process	GPU thread
Work distribution	1D row-wise domain decomposition	Grid-stride cell updates
Data dependency	Neighboring-process boundary rows	Neighboring cells in device memory
Primary overhead	Inter-process communication	Thread scheduling and occupancy constraints
Optimization	Overlap communication and computation	Tune kernel launch configuration
Observed optimum	64 MPI processes	8×8 thread block
Scaling limit	Communication dominates local work	Larger configurations reduce efficiency

7.1 MPI: Computation vs. Communication

MPI reduces local computation per process as the number of partitions increases. Initially, this reduces runtime because additional CPU processes execute useful work concurrently.

However, communication does not disappear as the local workload shrinks. Processes must continue exchanging partition boundaries every generation. Consequently, the ratio

$$\frac{\text{useful computation}}{\text{communication overhead}}$$

decreases as process count becomes large.

The substantial runtime increase observed at 128 processes indicates that the simulation has crossed this balance point for the tested workload.

7.2 CUDA: Parallelism vs. Resource Utilization

CUDA presents a different tradeoff. Threads operate on data already resident in GPU memory, avoiding explicit distributed-memory message passing. Performance is instead strongly affected by the organization of GPU work.

Small blocks can underutilize available parallelism, while larger blocks contain more resident threads per block and can reduce scheduling flexibility or occupancy depending on register usage and hardware limits.

The measured optimum at an 8×8 thread block illustrates why launch configuration is a performance parameter rather than merely a correctness parameter.

7.3 Shared Performance Principle

Despite their architectural differences, both experiments exhibit the same general shape:

$$\text{more parallelism} \rightarrow \text{better performance} \rightarrow \text{optimal point} \rightarrow \text{overhead dominates.}$$

For MPI, the dominant overhead is communication.

For CUDA, the dominant factors are GPU execution configuration and resource utilization.

Effective parallel programming therefore requires identifying the point at which additional parallelism no longer compensates for the overhead introduced by the execution model.

8. Limitations and Future Work

The existing experiments provide useful scaling observations, but several extensions would make the comparison more comprehensive.

First, the MPI and CUDA studies use different generation counts. MPI was evaluated for 500 generations while CUDA was evaluated for 100 generations. Consequently, their absolute execution times should not be interpreted as a direct CPU-versus-GPU performance comparison.

A controlled follow-up study could use identical simulation configurations across both implementations.

Second, the MPI experiment begins at four processes rather than including a single-process serial baseline. The current results therefore measure scaling relative to an already-parallel configuration rather than conventional parallel speedup.

A sequential baseline would enable calculation of

$$S_p = \frac{T_1}{T_p},$$

as well as parallel efficiency

$$E_p = \frac{S_p}{p}.$$

Third, CUDA performance could be studied more systematically by varying both launch-grid dimensions and non-square thread-block shapes, while collecting hardware-level measurements such as occupancy, memory throughput, register pressure, and Streaming Multiprocessor utilization.

Finally, larger grid sizes could reveal how the optimal MPI process count and CUDA launch configuration change as computational intensity increases. Larger problems may amortize communication and scheduling overhead more effectively, allowing stronger scaling than observed on the 512×512 workload.

9. Conclusion

This work explored two parallel implementations of Conway’s Game of Life using MPI and CUDA.

The MPI implementation distributed the grid through one-dimensional row decomposition and used non-blocking communication to overlap boundary exchange with local computation. Runtime improved as process count increased from 4 to 64, decreasing from approximately 0.0655 seconds to 0.0090 seconds, before communication overhead caused performance to degrade sharply at 128 processes.

The CUDA implementation mapped cell updates onto a two-dimensional GPU thread hierarchy using padded grid representations, custom kernels, and two-dimensional grid-stride execution. Tuning the square thread-block dimension reduced runtime from 0.0357 seconds with 1×1 blocks to 0.00451 seconds with 8×8 blocks, after which larger configurations again reduced performance.

The two experiments expose different bottlenecks but reinforce the same systems principle: parallel performance is determined not by maximizing the number of workers, processes, or threads, but by balancing useful computation against the overhead introduced by the underlying execution model.

For distributed-memory systems, that balance is largely governed by communication. For GPU systems, it is governed by how effectively computational work maps onto finite hardware resources. Understanding these tradeoffs is therefore essential for designing scalable parallel applications.